

Challenges in Migrating from RDBMS to NoSQL Databases

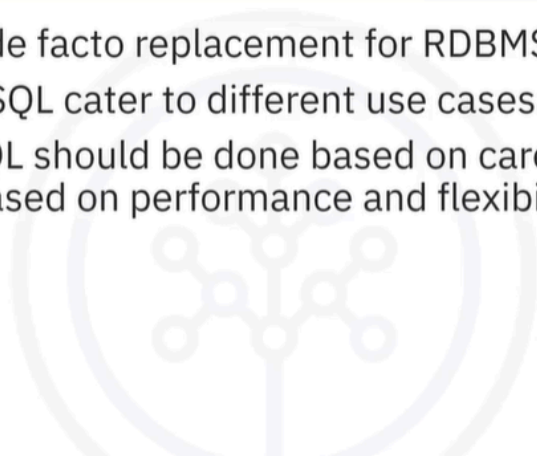
Sure! Let's talk about the differences between RDBMS (Relational Database Management Systems) and NoSQL databases in simple terms.

RDBMS is like a well-organized library where every book (data) has its specific place on the shelf (table), and you can easily find it using a catalog (queries). This system works best when you have structured data that needs to be consistent, like a library that keeps track of all its books, authors, and genres. On the other hand, NoSQL databases are more like a big storage room where you can throw in all kinds of items without worrying about how they are organized. This flexibility is great for unstructured data, like social media posts or sensor data, where you might not know exactly what you'll need to store.

In summary, if you need strict organization and consistency, RDBMS is your go-to. But if you want flexibility and scalability, especially with large amounts of unstructured data, NoSQL is the way to go.

RDBMS to NoSQL: Overview

- NoSQL is not a de facto replacement for RDBMS
- RDBMS and NoSQL cater to different use cases
- RDBMS to NoSQL should be done based on careful case-by-case analysis based on performance and flexibility requirements



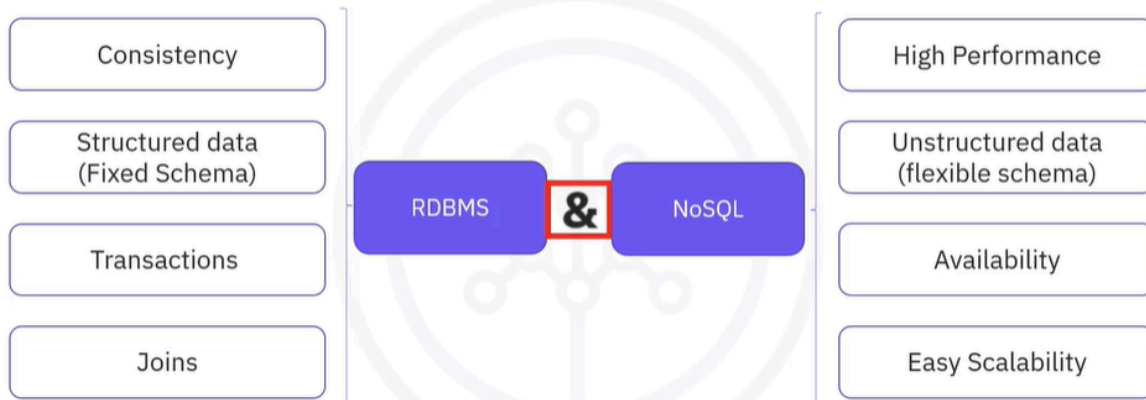
- There is a misconception sometimes that either NoSQL is the replacement of RDBMS or that you need to choose between relational and NoSQL databases. In reality, RDBMS and NoSQL databases are not competing because they cater to quite different requirements and use cases. While it is possible to migrate from an RDBMS system to a NoSQL system, this should be done based on the characteristics required by your end solution. The change might be driven by additional performance or flexibility, whatever your reasons are, you need to analyze them against what NoSQL can do for you.

RDBMS or NoSQL



- Let us see some generic situations where we may go either relational or nonrelational, meaning NoSQL if you need full consistency for your data, if your data is structured. If you need multi-document transactions and complicated joins, in this case, a relational database will make more sense if you have loads of data and need a high performance system.
- If your data is unstructured and could benefit from a flexible schema, if your system needs to be available and scalable, then in this case a NoSQL database will make more sense.

RDBMS and NoSQL



- There might be other cases in which both relational and NoSQL databases will be needed. If you have too much data and need performance and need to scale fast. But at the same time you also need transactions support and complex joins on your data than you might think of a combined solution.

RDBMS to NoSQL: A mindset change

- **Data-driven** model to **query-driven** data model
 - RDBMS: Starts from the data integrity, relations between entities
 - NoSQL: Starts from your queries, not from your data models, based on the way the application interacts with the data
- **Normalized** to **Denormalized** data
 - NoSQL: Think how data can be structured based on your queries
 - RDBMS: Start from your normalized data and then build the queries
- When embracing the NoSQL world and coming from a relational one, there are a few things you should pay attention to, like the fact that in NoSQL, your data model is driven by your queries, not the data itself. In a relational world, the solution design starts from the data, the entities and their relationship.

- In NoSQL, it's not the data that drives your data model or schema, it's the way your application accesses the data and the queries you are going to make. In NoSQL models should be based on how the app interacts with the data rather than how the model can be stored as rows in one or more tables. Another factor to pay attention to is the fact that in RDBMS, data is normalized while in NoSQL it is denormalized. With NoSQL, starting from your query means that you will structure your data on disk accordingly. Thus, you may need to store the same data in different models just to answer the question, and this will lead to data denormalization.
- While data in RDBMS is normalized, you need to be open to the situation in which you start with your queries instead of your data.

RDBMS to NoSQL: A mindset change

- **From ACID to BASE model**

- Choose availability versus consistency
- Apply the CAP Theorem: choose between availability and consistency
- Consider availability, performance, geographical presence, high data volumes

- **NoSQL systems, by design, do not support transactions and joins except in limited cases**

- When migrating from relational to NoSQL databases, you need to understand that sometimes services require availability more than consistency when both availability and performance are needed. Thus, distributed systems consistency cannot be ensured. Remember cap theorem, many of today's online services value availability more than consistency.
- And because of this they look for systems that can provide it, taking into consideration the amount of data they are dealing with and their geographical presence. One last thing to know when dealing with NoSQL databases, these are not designed to support transactions or joins or complex processing

except in limited cases. You need to consider this when moving from RDBMS to NoSQL.

Recap

In this video, you learned that:

- NoSQL systems are not a de facto replacement for RDBMS
- RDBMS and NoSQL cater to different use cases
- You could use both RDBMS and NoSQL
- A migration from RDBMS to NoSQL could be triggered by performance or flexibility requirements
- Migrating from RDBMS to NoSQL requires adoption of NoSQL concepts